Revision Notes

Last minute preparation for the mid term review. Numbers first, then the questions most likely to be asked and how to answer them.

Part 1: Developer (Rohith K Nair)

Numbers to know cold

Dataset	Rows	Columns	Classes
Documents	4,950	2 raw, 11,482 after vectorising	9, balanced at 550 each
Messages	3,360	3 raw, 1,838 after vectorising	8 intents, 3 sentiments
Journeys	6,000	12 features + 1 target	2, split 43.3 / 56.7

Total 14,310 examples. Split 75 / 25, stratified, random_state 42.

Document length: mean 348 characters. Difficulty tiers: clean 2,005, degraded 1,841, severe 1,104. Built from 21 templates.

Sparsity after vectorising: 99.26 per cent for documents. Features pruned by min_df of 2: from about 41,900 down to 11,482.

The one minute explanation of what you built

Three generators, one for each prediction problem. A preprocessing pipeline that turns text into weighted number vectors and rescales the numerical features. A stratified split with a leakage audit. Ten figures with written observations. That is the whole of your contribution to this phase.

Questions you will be asked

What is TF-IDF, in one sentence? A weighting that scores a word by how often it appears in a document against how rare it is across the whole corpus, so common words count for little and distinctive words count for a lot.

Why character n-grams as well as word n-grams? Because scan damage destroys whole words. A corrupted word becomes a completely different token, so the word based features fail. Most of the three to five letter sequences inside that word survive, so the character features still work. This is why the severe tier scores 0.937 instead of collapsing.

What does sublinear_tf do? It replaces the raw count with 1 plus the log of the count, so a word repeated twenty times in one document does not dominate its vector.

Why min_df equals 2? A term appearing in only one document cannot generalise to a new document. It can only help the model memorise that one example.

Why scale the numerical features? They use different units. One column is a percentage, another is a day count, another is bounded between minus one and one. Without scaling the column with the largest numbers dominates simply because its numbers are bigger, and the model coefficients stop being comparable.

Why stratified rather than a plain random split? Stratification keeps each class proportion the same in both halves. With a plain random split a smaller class can be unlucky and end up underrepresented in the test fold, which makes its score unstable.

Why random_state 42? Any fixed number works. The point is that the split is reproducible, so anybody rerunning it gets the same folds. Test TC-15 asserts this.

Why did you not impute missing values? There are none. We checked every column and every record across all three datasets and the count is zero. That is a consequence of generating the data rather than collecting it, and it is one way our data is easier than real data.

Why did you keep the outliers? The features that produce outliers are counts and waiting times. An employee inactive for twenty five days is exactly the case the system is meant to catch. Removing that row would delete the problem we are trying to detect.

The three findings, in order of strength

1. **The first model scored 100 per cent and we treated that as a fault.** Perfect accuracy

with zero cross validation variance means the task was too easy, not that the model was good. We rebuilt the generator with three damage tiers until the hardest tier read 0.937.

1. **Three features turned out to be one feature.** Stage, progress and percentage of tasks

complete correlate above 0.90 with each other, reaching 0.97 for the strongest pair. This explained why removing any one of them barely changed the result.

1. **We were wrong about our own leakage.** Explained in the Tester section. If asked, hand

to Prisha.

Traps to avoid

Do not quote 0.986 as the document classifier result. Quote **0.937**, the severe tier figure, and explain why. Clean scans score a perfect 1.000 and that tells you nothing.

Do not present the sentiment figure of 0.996 as a capability. It is high because the generator marks sentiment with stock phrases. Say that before anyone asks.

Do not say the models are evaluated. They are trained but formal evaluation is next phase.

Part 2: Team Lead (Kritika Sah)

What you own

The problem, the objectives, the scope, the workflow and the overall picture. You open and you close.

Key points

The problem in one sentence. Onboarding is manual, document heavy and slow, nobody notices when a new hire stalls, and existing systems are checklists that record steps without reading documents or predicting problems.

Who is affected. The new hire, because a bad first fortnight is hard to recover from. HR, because the work is repetitive. The company, because replacing somebody who leaves in year one costs six to nine months of salary.

The five objectives. Build and describe the datasets. Preprocess with justification. Analyse before modelling. Split without leakage and verify it. Then develop and evaluate models, which is next phase.

The six step method. Collection, understanding, preprocessing, splitting, then model implementation and evaluation. Steps one to four are complete.

Who did what. Rohith wrote the code and the analysis. Prisha wrote 34 test cases and ran the leakage measurement. Pasham owned data quality, verification and the defect register. You handled problem definition, objectives, scope and coordination.

The thing to say in the first thirty seconds

That this review covers work up to data splitting, and that no accuracy or F1 figures will be shown because that belongs to the next phase. Saying it early stops anyone wondering why they are missing.

If asked what you would do differently

Put the project under version control from day one. There is no commit history and the work exists on one machine. It is the least technical item on the list and probably the most important.

Part 3: Tester (Prisha Saini)

Numbers

34 test cases, 55 executed checks, all passing, about twenty seconds, no network needed.

Categories: 7 data validation, 3 preprocessing, 4 split integrity, 4 functional, 3 boundary, 3 exceptional, 2 access control, 2 regression, 2 performance, 1 security.

The leakage story, which is your strongest material

70 messages appear in both the training and the test fold. That is 8.3 per cent of that test set. Short messages like a plain thank you get generated identically by different random draws and a random split cannot separate them.

The expectation is that this inflates the score. You measured it instead of assuming.

Accuracy on the leaked rows: **0.9868**. On the clean rows: **0.9974**. The model did **worse** on the rows it had already seen.

The explanation is that the duplicated messages are the short ambiguous ones, which are the hardest to classify anyway. The duplication is concentrated in the difficult cases, so it drags that score down rather than lifting it.

The defect is still real and still needs fixing. The explanation for it was wrong, and that was written down rather than deleted.

Design decisions to defend

No test calls a language model. The assistant phrases answers differently every run, so a test comparing text would fail for reasons unrelated to the change being tested, and the team would learn to ignore it. Only deterministic parts are tested.

All file tests write to a temporary folder. Running a test suite must never change the state of the project. An early version added a row to the working database, which was unacceptable.

Two tests assert a defect exists. They confirm the duplicate problem and set an upper bound. If it grows the test fails. If somebody fixes it silently the test also fails. A known defect with no test around it can expand unnoticed.

Example of a failing test that was not a defect

The sparsity threshold in TC-09 was set at 99 per cent from a figure in the data report, and the test failed. The report figure was for word features alone while the code combines word and character features, which are legitimately denser. The code was right and the threshold was wrong. It was corrected to 95 per cent with a comment explaining why.

What is not covered

No API level tests. No authentication tests, because there is no authentication. No frontend tests. No assessment of whether an assistant answer is good. No automatic accuracy threshold once models exist.

Part 4: QA (Pasham Srikar)

What the role is, and how it differs from testing

Testing asks whether the system does what was specified. Quality assurance asks whether the way of working can produce something trustworthy, and whether the claims being made are actually supported by evidence. The Tester finds faults in the system. QA finds faults in the process and in the reasoning.

The five quality criteria

Data is fit for use. Preprocessing is verified. Claims are supported. Defects are visible. Work is reproducible.

Data quality results

Missing values: zero in all three datasets. Empty records: zero. Zero variance features: zero. Class balance ratio 1.00 for documents and intents.

The failure: 200 exact duplicates in the message corpus, 293 after normalising case and whitespace.

Outliers across the twelve numeric features were found and deliberately **retained**, because the features producing them are counts and waiting times whose long tail is the signal of interest.

The defect you found that no test would have caught

Two features in the risk model, `response_latency_hours` and `is_remote`, were hardcoded to fixed values at prediction time while the model had been trained on values that varied. Learned weights were being applied to information the model never actually received.

Nothing crashed. No test failed. Every individual component worked correctly. It was found by reading the prediction path line by line and asking what each value actually was.

Neither could be computed from the available data, so both were removed and the model retrained. The reported score barely moved, which is the point. It now describes what the model genuinely sees.

Traceability

Every number in the slides, the paper and the reports comes from one of four generated files. Nothing is typed by hand.

- `python -m ml.data_report` produces the dataset figures
- `python -m ml.eda` produces the figures and their observations
- `python -m tests.make_test_report` produces the test results
- `python docs/build_midterm_deck.py` produces the slides

The five recommendations, in order

Version control first. Deduplicate the messages before splitting. Add accuracy thresholds to the suite once models exist. Obtain a real document corpus. Merge or drop the redundant progress features.

Shared: questions anybody might get

Is this real data? No. All three datasets are generated, for two separate reasons. Onboarding documents are identity and financial records that cannot lawfully be collected for a student project. And records of who struggled during onboarding do not exist because the system has never been deployed. State this before being asked.

How do you know your generator is any good? We do not claim it is. We claim it produces a task hard enough that the resulting numbers mean something. Evidence: the severe tier scores 0.937 rather than 1.000, the intent classifier sits at 0.937 because the corpus contains genuinely unresolvable messages, and the journey generator includes a component in the outcome that no feature observes, so perfect prediction is impossible by construction.

Why not use a pre-trained model like BERT? Everything had to run offline on a laptop with no GPU and no model download. That constraint drove the choice of classical methods. Moving to a transformer is in the future work.

What is your biggest limitation? The data is synthetic, so every figure measures consistency with our own generators rather than real world performance. Replacing the document corpus with a public one such as RVL-CDIP is the first item of future work.

What would you fix first if you had another week? Version control, then deduplicating the message corpus, then a real document corpus.